

Scalability & Future Plan

Date	06 July 2026
Team ID	Not applicable / not assigned
Project Name	Credit Card Approval Prediction
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority
1	Label encoders used during training are not saved alongside model.pkl, so the web form's numeric inputs must manually match the training-time encoding order	Risk of incorrect predictions if the encoding used in app.py drifts from what main.py produced during training	High
2	Model evaluation relies only on Accuracy and F1-score; class imbalance between Good/Bad customers is not explicitly measured or addressed (no ROC-AUC, class weighting, or resampling)	The model may look accurate overall while still performing poorly on the minority 'Bad Customer' class, which matters most for credit risk decisions	High
3	No requirements.txt, README, or production configuration exists yet, and the Flask app currently runs with debug=True	Harder for others to reproduce the environment, and debug mode is unsafe if the app is exposed publicly	Medium

Scalability Plan:

S. No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Single-instance local Flask development server (app.run()); handles one request thread at a time	Deploy on Render behind a production WSGI server (e.g. gunicorn) with multiple workers, and add a load balancer / autoscaling if concurrent traffic grows
2	Data Storage	Static local CSV files (application_record.csv, credit_record.csv) loaded directly by the training script	Migrate applicant and credit-history data into a managed database (e.g. PostgreSQL) so new records can be added incrementally without re-loading flat files
3	Performance	Model is loaded once and predictions run synchronously per request with no caching or batching	Keep the model cached in memory (as now), add lightweight request logging/monitoring, and consider batching or async processing if prediction volume increases significantly
4	Security	No authentication on the form, minimal input validation, and debug=True enabled in the Flask app	Disable debug mode for production, add server-side input validation, serve over HTTPS (provided by Render), and consider rate-limiting the /predict endpoint

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
-------	-------------------------------	-----------------	-----------------

Phase 1	Add requirements.txt and README; save the label-encoding pipeline alongside model.pkl (e.g. via scikit-learn Pipeline/ColumnTransformer)	2–4 weeks	Reproducible setup for evaluators/collaborators and safer, consistent inference
Phase 2	Complete public deployment on Render with debug=False and a production WSGI server	Within 1 month	Publicly accessible demo URL for evaluators and real users to test
Phase 3	Improve model evaluation with ROC-AUC/precision-recall and address class imbalance (e.g. class_weight='balanced' or SMOTE)	2–3 months	More reliable detection of 'Bad Customer' cases despite the imbalanced dataset
Phase 4	Migrate to a database-backed pipeline with an admin dashboard for monitoring predictions and retraining the model periodically	3–6 months	Scalable to real institutional use, with ongoing model maintenance and drift monitoring